

@andrian.yablonskyy/thub-agent

The Agent CLI (**thub**) for **TestHub** — a self-hosted job network that lets CI/CD pipelines and individual developers run firmware tests on real hardware or emulators in a private lab. **thub** is the single entry point for both: it's stateless, everything it knows comes from the **Coordinator** API, and it runs identically on a GitHub-hosted runner and a developer laptop — a developer reproducing a CI failure runs exactly the same command the pipeline runs.

See the [main TestHub repo](#) for the full system architecture and how this fits with the Coordinator and the [Client](#).

Install

```
npm i -g @andrian.yablonskyy/thub-agent
# or, one-off in CI:
npx -y @andrian.yablonskyy/thub-agent run --type sw --download-
file "$IMAGE_URL" --git-repo "$TESTS_REPO" --command ./ci/test.sh
--wait
```

Configuration

Read from flags, then environment (**THUB_URL**, **THUB_TOKEN**, **THUB_GROUP**, **THUB_USER**), then `~/.config/thub/agent.json`, then a bundled default. **url/token** are required by the time a command actually talks to the Coordinator; **group/user** are optional everywhere.

`npm install -g` creates `~/.config/thub/agent.json` for you (blank **url/token**, so nothing works until you set them) if it doesn't already exist — a re-install never overwrites it. Fill it in with **thub config set**:

```
thub config set url https://thub.example.com
thub config set token agt_...
thub config set group 548ae4ae... # optional default --group
thub config set user "Your Name" # optional default --user
```

Commands

<code>thub run</code>	<code>[options]</code>	Submit a test job and follow its log
<code>thub status</code>	<code><jobId></code>	Show status; follow log if running, show artifacts if done
<code>thub cancel</code>	<code><jobId></code>	Cancel a job
<code>thub resources</code>		List resources and their status
<code>thub jobs</code>	<code>[--mine]</code> <code>[--state <s>]</code>	List recent jobs
<code>thub config</code>	<code>set <key> <value></code>	Save coordinator URL / token / default group / default user locally

thub check-update	Compare this Agent with the
latest published version	
thub self-update [--to <x.y.z>]	Update this Agent with npm
i -g	
thub --version	

Self-update. An admin can request an update for this agent (or all agents) on the Coordinator's Agents page. The next command that talks to the Coordinator then installs it with `npm i -g` (retrying through `sudo` on an interactive terminal) and re-runs itself on the new version. If the install fails — e.g. no permission in CI — it prints the manual command and carries on with the current version; a run never fails because of an update. Set `THUB_NO_SELF_UPDATE=1` to opt out.

Key options for `thub run`:

Option	Description
<code>--type hw sw</code>	Required resource type.
<code>--board <name> / --label <l></code>	Required labels (repeatable).
<code>--group <groupId></code>	Restrict scheduling to resources that are members of this group. Falls back to <code>THUB_GROUP</code> / <code>thub config set group <id></code> .
<code>--client <name id></code>	Run on this specific Client (resource name or id) only; the job waits in that Client's queue even if other matching resources are idle.
<code>--user <name></code>	Free-text job owner — a label, not an identity. Falls back to <code>THUB_USER</code> / <code>thub config set user <name></code> .
<code>--command <string></code>	Required. The task's entry point: a shell command the Client runs (<code>sh -c</code>) in the task's work directory — the <code>--git-repo</code> checkout, else an empty directory — after preparing its inputs. Its exit code is the verdict. On HW it flashes the board itself (the Client doesn't); it gets <code>THUB_DUT_STLINK/_UART/_USB/_HOST/_CONTAINER</code> , <code>THUB_DOWNLOAD_<n></code> , <code>THUB_DOWNLOADS_DIR</code> , <code>THUB_GIT_COMMIT</code> , <code>THUB_SUITE</code> , <code>THUB_META_*</code> .
<code>--download-file <url></code>	A file the Client downloads before running the command (repeatable, <code>http(s)</code>), into the job's <code>downloads/</code> directory.
<code>--docker-image <name></code>	SW only: a Docker image the Client runs as the DUT instead of its own <code>sw.image</code> (the Client must allow it: <code>sw.allowJobImages</code>).
<code>--git-repo <url> [<branch> <tag> <commit>]</code>	A git repository the Client clones (default ref: the default branch); the command runs in the checkout.

Option	Description
<code>--depth <n></code>	With <code>--git-repo</code> : commits to fetch, default <code>1</code> ; <code>0</code> = full history.
<code>--suite <name></code>	Passed to the command as <code>THUB_SUITE</code> .
<code>--arg <value></code>	Extra argument for the command, as <code>"\$@"</code> (repeatable).
<code>--timeout <dur></code>	e.g. <code>30m</code> , default <code>30m</code> .
<code>--priority <n></code>	0–100; CI defaults to 50, CLI to 60 so a developer is not starved by a busy pipeline.
<code>--meta <key=value></code>	Arbitrary metadata stored on the job (repeatable) — CI job ids, git coordinates, anything else worth attaching to the run.
<code>--dry-run</code>	Exercise the full pipeline without the Client executing anything for real.
<code>--wait</code>	Do not detach on job end; exit with the job's verdict code (used in CI).
<code>--detach</code>	Print the job id and exit immediately.
<code>--json</code>	Machine-readable output.

- `thub run` prints the **job ID**, then streams logs until Ctrl-C. Ctrl-C detaches; the job keeps running on the Client.
- `thub status <jobId>` prints the current state; if active it keeps streaming, if done it prints the verdict and artifact download links.

Exit codes make the Agent usable as a CI step:

Code	Meaning
<code>0</code>	<code>PASSED</code>
<code>1</code>	<code>FAILED</code>
<code>2</code>	<code>ERROR</code> , <code>TIMEOUT</code> , or <code>LOST</code>
<code>3</code>	<code>CANCELED</code>
<code>4</code>	Usage, auth or connection error
<code>130</code>	Detached with Ctrl-C (job still running)

Examples

Run a task — download the firmware, check out the tests at a tag, flash and test (HW):

```
thub run --type hw --board nucleo-f401re \
  --download-file "$IMAGE_URL" \
  --git-repo https://github.com/yourorg/firmware-tests.git v1.4.0 \
```

```
--command 'st-flash --serial "$THUB_DUT_STLINK" --reset write
"$THUB_DOWNLOAD_1" 0x08000000 && ./ci/test.sh "$@"' \
--arg --junit --wait
```

SW task in a Docker image of your own:

```
thub run --type sw --docker-image alpine:3.20 \
  --git-repo git@github.com:yourorg/firmware-tests.git main --
depth 20 \
  --command 'make test' --wait
```

Associate a CI/CD job id with the internal job id:

```
thub run --type sw --download-file "$IMAGE_URL" --git-repo
"$TESTS_REPO" --command ./ci/test.sh \
  --meta ciJobId="$GITHUB_RUN_ID" --wait
thub status A-00123 --json | jq '.spec.meta.ciJobId'
```

Pass extra metadata — `--meta key=value` reaches the command as `THUB_META_<KEY>` (`ciJobId` → `THUB_META_CI_JOB_ID`).

Dry-run the pipeline — proves the Coordinator↔Client plumbing works without real hardware, a real emulator image, or reachable downloads:

```
thub run --type sw --download-file https://does-not-
exist.invalid/app.bin \
  --command ./ci/test.sh --dry-run --wait
```

Run on a specific resource group only:

```
thub run --type sw --git-repo "$TESTS_REPO" --command ./ci/test.sh
\
  --group 548ae4ae-ac5b-401f-acaa-24bbe790e62d --wait
```

Label a job with its owner — purely informational, shows up on the dashboard, in `thub jobs`, and on the Client's own console:

```
thub run --type hw --board nucleo-f401re \
  --git-repo "$TESTS_REPO" --command ./ci/test.sh --user "Your
Name" --wait
```

GitHub Actions

```
test-sw:
  needs: build
  runs-on: ubuntu-latest
  env:
    THUB_URL: https://thub.example.com
    THUB_TOKEN: ${ secrets.THUB_AGENT_TOKEN }
  steps:
    - run: |
        npx -y @andrian.yablonsky/thub-agent run --type sw \
          --download-file "${ needs.build.outputs.image_url }" \
          --git-repo "$GITHUB_SERVER_URL/$GITHUB_REPOSITORY.git"
          "$GITHUB_SHA" \
          --command ./ci/sw-tests.sh --suite full --wait
```

If the GitHub job is canceled, the runner sends **SIGINT** to the Agent. In **--wait** mode (CI), that's treated as a cancel request (**POST /jobs/:id/cancel**) before exiting, so abandoned CI jobs don't hold hardware. In interactive mode, Ctrl-C only detaches.

Development

```
npm install
npm run lint
```

License

Proprietary — see the header comment in each source file.